Projekt I - Klasyfikacja cyfr (2026)

Wstęp

Celem ćwiczenia jest zbudowanie systemu automatycznej klasyfikacji cyfr wypowiadanych przez różnych mówców oszacowanie jego skuteczności wyrażonej miarą Accuracy (%) :

- wraz z wartością odchylenia standardowego wyniku dla testu krzyżowego (kroswalidacji)
- jednej liczby dla zbioru ewaluacyjnego.

Dane

Pliki zawierające nagrania cyfr do treningu/testowania można znaleźć na stronie kursu. Są to nagrania cyfr artykułowanych przez różne głosy. Każdy głos wypowiada daną cyfrę jednokrotnie. Wypowiedziana cyfra zawarta jest między *podkreślnikami* w nazwie każdego pliku.

Algorytmy

- Można wykorzystać dowolny rodzaj parametryzacji. Sugerowane: MFCC. Dla osób chętnych można skorzystać z dowolnego ekstraktora cech opartego na głębokich modelach takich jak Wav2Vec2 (https://huggingface.co/docs/transformers/tasks/audio_classification)
- Można wykorzystać dowolne modelowanie i klasyfikator. Sugerowane modele i klasyfikatory to: GMM, Sieć Neuronowa (MLP, Conv1d), SVM, kNN, Decision Tree, Random Forest (przykłady i implementacje w https://scikit-learn.org/stable/supervised_learning.html#supervised-learning).

Wskazówki organizacyjne

- W pierwszej kolejności należy osiągnąć jak najszybciej działający system (zestaw funkcji i/lub zapisanych modeli) umożliwiający określenie klasy “nieznanego” nagrania. Stanowi to tak zwany Minimum Viable Product (MVP).
- Wybierając klasyfikator należy przemyśleć typ i złożoność klasyfikatora (np. głębokość i szerokość warstw sieci neuronowej, liczbę komponentów GMM itp.) w kontekście posiadanych danych (liczba klas, ilość obserwacji w zbiorze treningowym, jakość nagrań itp.).
- Warto zaprojektować system, w którym w czasie uruchomienia można zadeklarować lub łatwo zmodyfikować (np. jako argument funkcji uruchamiającej cały system lub ustawienie wartości w pliku konfiguracyjnym) najważniejszych parametrów (np. długość ramki, liczbę filtrów melowych, rząd predyktora, liczbę współczynników cepstralnych, liczbę komponentów GMM, typ macierzy kowariancji, liczbę permutacji testu krzyżowego - jeśli dotyczy, i inne...).
- Bardzo ważne jest przemyślenie i opisanie dokładne struktury przepływu danych w systemie. Np jakich typów użyć by łatwe było utworzenie macierzy zawierającej cechy z wszystkich dostępnych wypowiedzi pojedynczej cyfry dla wybranych mówców. W szczególności warto zaplanować pojedynczą strukturę danych zawierającą cały zbiór cech. Zapobiegnie to później np wielokrotnemu kopiowaniu danych podczas tworzenia struktur na potrzeby testów krzyżowych.
- Test krzyżowy w tym projekcie polega na usunięciu 20% losowych mówców (nie losowych plików tych mówców) ze zbioru treningowego i weryfikacji skuteczności na ich podstawie (próbki testowanych mówców nie biorą udziału w treningu). Ma to na celu zmniejszenie wpływu podobieństwa głosu na pomiar skuteczności systemu.

Wskazówki implementacyjne

- Warto wykorzystać keyworded arguments w Pythonie, by wygodnie zmieniać parametry w czasie eksperymentów.
- Poruszanie się po katalogach (i nazwach plików) można zrealizować posługując się pakietem *os (https://docs.python.org/3/library/os.html)* lub *glob* (https://docs.python.org/3/library/glob.html)

- Pomocna może być możliwość zapisu zbioru sparametryzowanych plików do pojedynczego pliku (serializacja), co w dalszych etapach projektu może przyspieszyć prace na optymalizacją modelu. Zapis/odczyt do plików można zrealizować m.in. za pomocą pakietu *pickle (https://docs.python.org/3/library/pickle.html)*.

Zaliczenie zadania

- System realizowany będzie w trakcie 3 kolejnych spotkań laboratoryjnych.
- Proponowany plan działań
  - **Lab 1** - MVP: architektura systemu (opis przepływu danych), plan podziału zadań na najbliższy tydzień, wybór metod i bibliotek, wczytanie plików i odkodowanie ich nazw, parametryzacja całego zbioru oraz stworzenie podstawowego klasyfikatora.
  - **Lab 2** - Implementacja pętli testu krzyżowego (sklearn.model_selection.KFold.split) - pętli treningu/testowania, implementacja treningu klasyfikatora, szacowanie skuteczności klasyfikatora + dogranie zbioru ewaluacyjnego
  - **Lab 3** - Implementacja treningu klasyfikatora c.d., wykonanie testów i optymalizacja parametrów klasyfikatora, test na danych ewaluacyjnych.
- Praca powinna być wykonywana w stałych grupach od 2 do 4 osób.

Ocena prac

- Oceniana będzie systematyczność w pracy nad systemem (z tygodnia na tydzień) weryfikowana na podstawie cotygodniowych raportów zamieszczanych bezpośrednio w notatniku, w którym realizowany jest projekt (w postaci tekstu). Raporty powinny zawierać opisy takie jak: koncepcja systemu, zadania zrealizowane, zadania do zrealizowania, analizę problemów napotkanych w trakcie pracy, wnioski uzyskanych wyników, cząstkowe rezultaty, wykresy pośrednich etapów działania itp.
- Linki do notatników lub repozytorium na GitHub należy przesłać korzystając z Uczelnianej Platformy e-Learningowej przed rozpoczęciem się kolejnych zajęć laboratoryjnych w odpowiednich Zadaniach.
- Notebooki/pliki mają być redagowane w sposób przyrostowy z wyraźnym oddzieleniem kolejnych tygodni nie jako kolejne nowe pliki.
- Oceniana będzie prawidłowość dobranych metod i/lub ich ciekawe i racjonalne uzasadnienie,
- Brane pod uwagę będą również wartości i jakość uzyskanych wyników ostatecznych (RR, macierze błędów) i konstruktywne uzasadnienie ich wartości.
- Przed ostatnimi zajęciami zostanie opublikowany ewaluacyjny zbiór plików o zawartości podobnej do tej ze zbioru treningowego. Pliki nie będą opisane. Opracowany system powinien więć zapisać plik csv zawierający w kolejnych liniach wpisy postaci:*nazwa pliku, rozpoznana cyfra, wartość likelihood*. Przykładowa zawartość pliku csv:

```
001.wav,9,-35.87
002.wav,2,-73.89
003.wav,2,-32.99
004.wav,3,-94.24
005.wav,7,-94.20
006.wav,5,-68.69
007.wav,2,-31.14
008.wav,7,-49.07
```

- Plik posłuży do oceny jakości systemu na zupełnie obcych danych (niedostępnych w czasie treningu i testów krzyżowych).

System “podstawowy”

- Skuteczność (*classification accuracy*, stosunek poprawnie zaklasyfikowanych plików do wszystkich plików) w kroswalidacjach dla systemu opartego o podstawowe 13 cech MFCC z 8-komponentowymi GMM to około **90%**. Na danych ewaluacyjnych taki system otrzymuje wynik około **75%**. Dane ewaluacyjne zawierają nagrania kobiet, dzieci i innych studentów Inżynierii Akustycznej.

- Pytania kontrolne:
    - Jaki wpływ na skuteczność ma liczba komponentów w użytych modelach GMM?
    - Jaki wpływ na skuteczność ma liczba współczynników MFCC?
    - Jakie klasy są między sobą najczęściej mylone?
    - Który element systemu ma największy wpływ na skuteczność klasyfikatora?
    - Jakie cechy sygnału wpływają na skuteczność systemu? Czy zbudowany klasyfikator jest uniwersalny? Czy można go bezpośrednio zastosować np w systemach IVR)?